Kelompok 6

Heuristic **Algorithm**

- Ahmad Idza Anafin - 5027241017
- Andi Naufal Zaki - 5027241059
- Rayhan Agnan Kusuma 5027241102
- Zidny Ilman Nafi'an - 5027221072



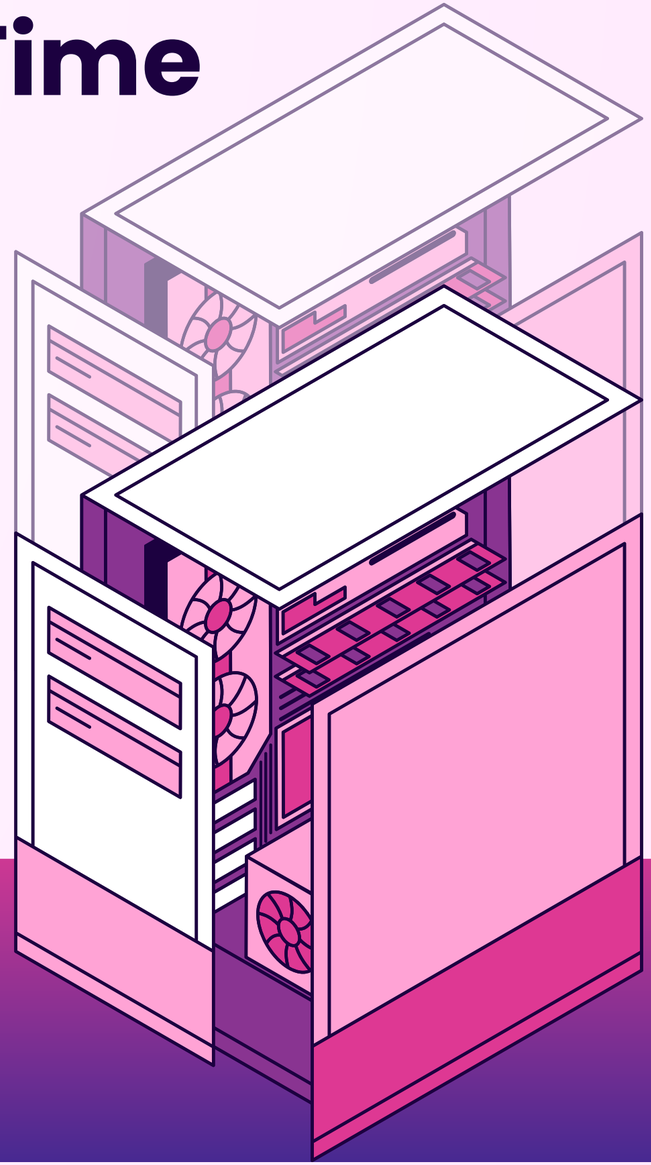
Heterogeneous Earliest Finish Time

Dua Tahapan Utama HEFT:

- Task Prioritization (Prioritas Tugas): Menghitung nilai peringkat (upward rank) untuk setiap task guna mengurutkannya dari prioritas tertinggi ke terendah.
- Processor Selection (Pemilihan Prosesor/VM): Memilih VM terbaik untuk setiap task berdasarkan waktu mulai paling awal (Earliest Start Time / EST) dan waktu selesai paling awal (Earliest Finish Time / EFT).

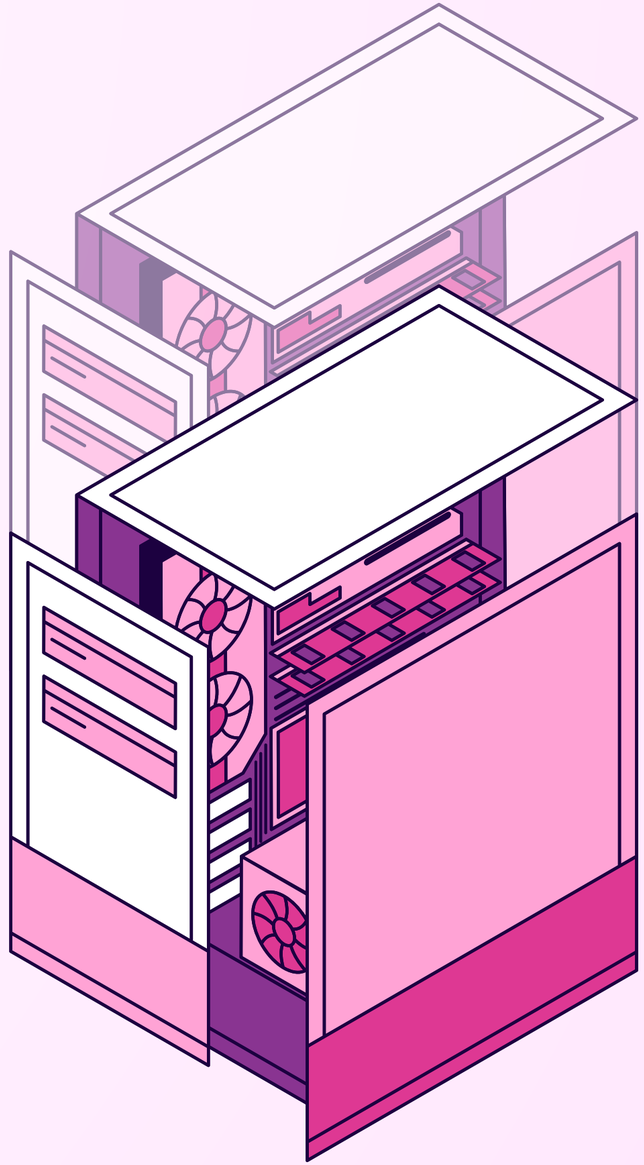
https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=993206

bertujuan menentukan urutan task dan resource/VM sehingga task selesai seawal mungkin dengan mempertimbangkan dependency, computation cost, dan communication cost.



iMPLEMENTATION

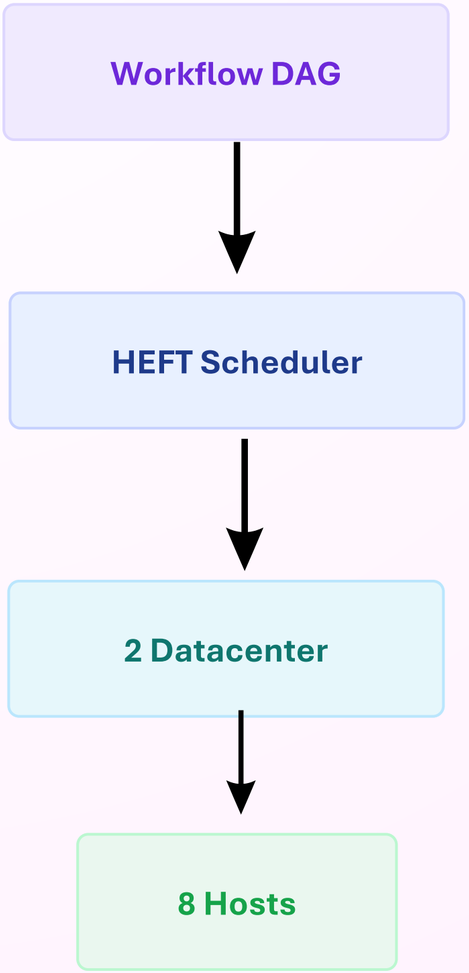




Kenapa HEFT?

Desain Project Tugas 3	Hubungan dengan HEFT
Workflow berbentuk DAG	HEFT mempertimbangkan predecessor / successor
2 datacenter + 8 host heterogen	HEFT dirancang untuk heterogeneous resources
10 / 20 / 40 VM	HEFT mengevaluasi alternatif resource
Computation cost berbeda	Digunakan dalam task prioritization & EFT
Communication cost	Masuk ke perhitungan upward rank / start time
Target utama: makespan	Processor dipilih berdasarkan earliest finish time

Environment Scheduling



VM-A
1 vCPU
2 GB

VM-B
2 vCPU
4 GB

VM-C
4 vCPU
8 GB

VM-D
2 vCPU
8 GB

Jumlah VM

10 / 20 / 40 VM per skenario

Task Scale

100 / 250 / 500 task untuk synthetic DAG

Workload

Synthetic DAG, Montage, CyberShake

Input yang Dibutuhkan HEFT

Informasi workflow dan resource sebelum scheduling dimulai

INPUT

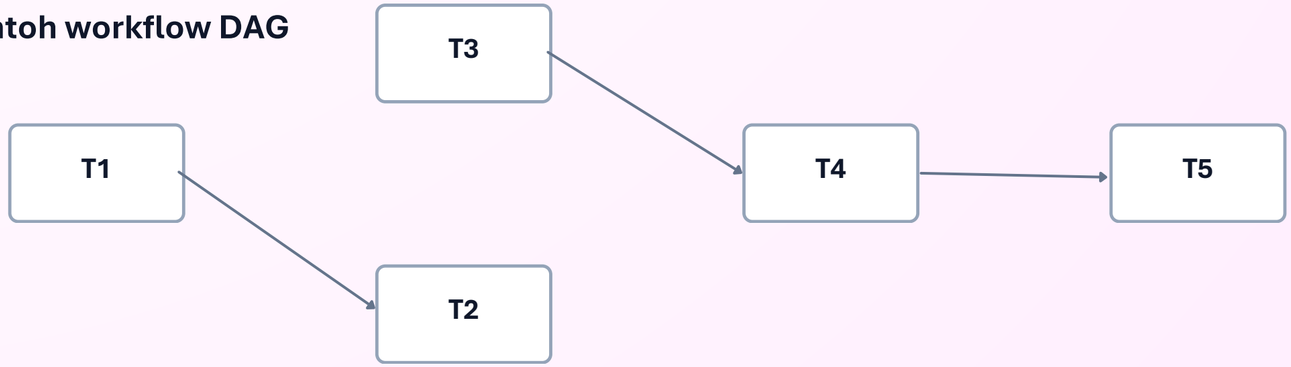
A. Workflow Information

- Task dan dependency
- Parent / predecessor
- Child / successor
- Computation cost
- Communication cost

B. Resource Information

- Daftar VM / processor
- Processing capability
- VM availability
- Execution time task pada tiap VM

Contoh workflow DAG



Untuk setiap task, HEFT perlu tahu “biaya” task pada resource berbeda.

```
executionTime[T2, VM1] = 20
executionTime[T2, VM2] = 12
executionTime[T2, VM3] = 18
```

Step 1 — Average Computation & Communication Cost

HEFT mengestimasi cost rata-rata sebagai dasar task prioritization

$$\bar{w}_i = \left(\sum_k w_{i,k} \right) / m$$

Average computation cost

Contoh T1

VM1 = 10
VM2 = 8
VM3 = 15

→ $\bar{w}_1 = (10 + 8 + 15) / 3 = 11$

$$\bar{c}_{i,j} = \text{average communication cost}$$

Dipakai untuk dependency edge $T_i \rightarrow T_j$

1. Ambil execution time tiap task pada seluruh VM
2. Hitung rata-rata computation cost
3. Hitung average communication cost pada edge

Catatan: nilai rata-rata ini dipakai untuk menentukan prioritas; VM final belum dipilih pada tahap ini.

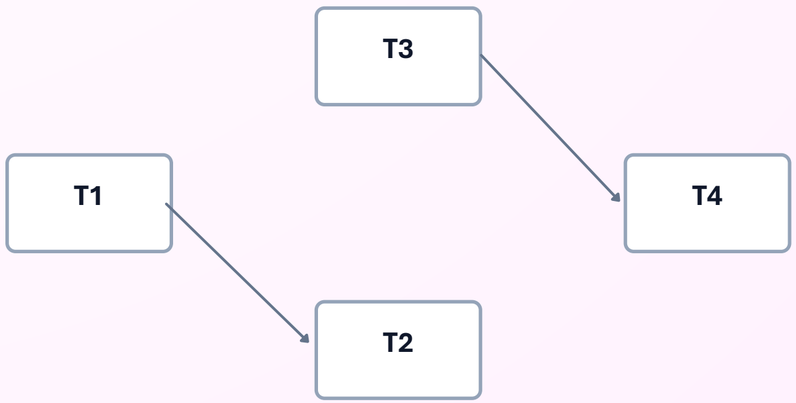
Step 2 — Upward Rank

Mengubah struktur DAG menjadi task priority

TASK PRIORITIZATION

$$\text{rank}_u(i) = \bar{w}_i + \max_j [\, \bar{c}_{i,j} + \text{rank}_u(j) \,]$$

Untuk exit task: $\text{rank}_u(i) = \bar{w}_i$



Perhitungan dari exit → entry

Intuisi Perhitungan

- Exit task dihitung terlebih dahulu
- Parent mempertimbangkan successor
- max() memilih jalur successor dengan kontribusi terbesar
- Hasil akhir menjadi prioritas scheduling

Semakin besar rank_u → semakin tinggi prioritas task.

Step 3 — Build Task Priority List

Mengurutkan task berdasarkan upward rank

Task	$rank_u$	Priority
T1	42	1
T2	35	2
T3	28	3
T4	19	4
T5	10	5

Sort descending

T1 → 42

T2 → 35

T3 → 28

T4 → 19

T5 → 10

Priority List
T1 → T2 → T3 → T4 → T5

Priority list menentukan urutan scheduling, bukan VM final.

Step 4 — Earliest Start Time (EST)

Menentukan kapan task dapat mulai pada VM tertentu

VM SELECTION

$$EST(i,k) = \max(Avail(k), \max_p [AFT(p) + c_{p,i}])$$

EST mempertimbangkan dua sumber keterlambatan

1 • VM Availability

Kapan VM tersedia setelah task sebelumnya selesai?

2 • Dependency

Parent selesai + communication delay ke child.

max(availability, dependency)

Task hanya boleh mulai setelah seluruh dependency terpenuhi.

Dependency constraint: $\text{Start}(T_j) \geq \text{Finish}(T_i) + \text{CommunicationTime}_{i,j}$

Step 5 — Earliest Finish Time (EFT)

Memilih VM yang membuat task selesai paling awal

VM SELECTION

$$EFT(i,k) = EST(i,k) + ExecutionTime(i,k)$$

Contoh evaluasi T2

VM	EST	Execution	EFT
VM1	20	15	35
VM2	10	30	40
VM3	15	12	27

Minimum EFT
27 → VM3

HEFT memilih:

VM3

karena EFT paling kecil

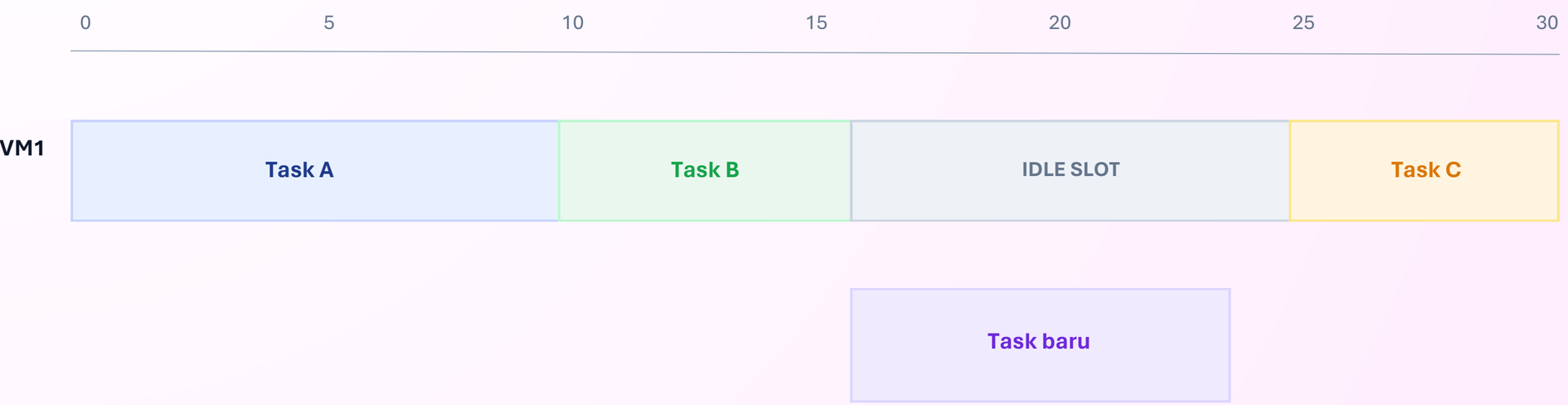
Inti keputusan: pilih $k = \text{argmin } EFT(i,k)$

Insertion-Based Scheduling

HEFT dapat memanfaatkan idle slot untuk menempatkan task lebih awal

SCHEDULE CONSTRUCTION

Contoh timeline VM



Langkah insertion

1. Cari gap kosong
2. Cek durasi task muat
3. Cek dependency / EST
4. Jika feasible, masukkan task ke gap

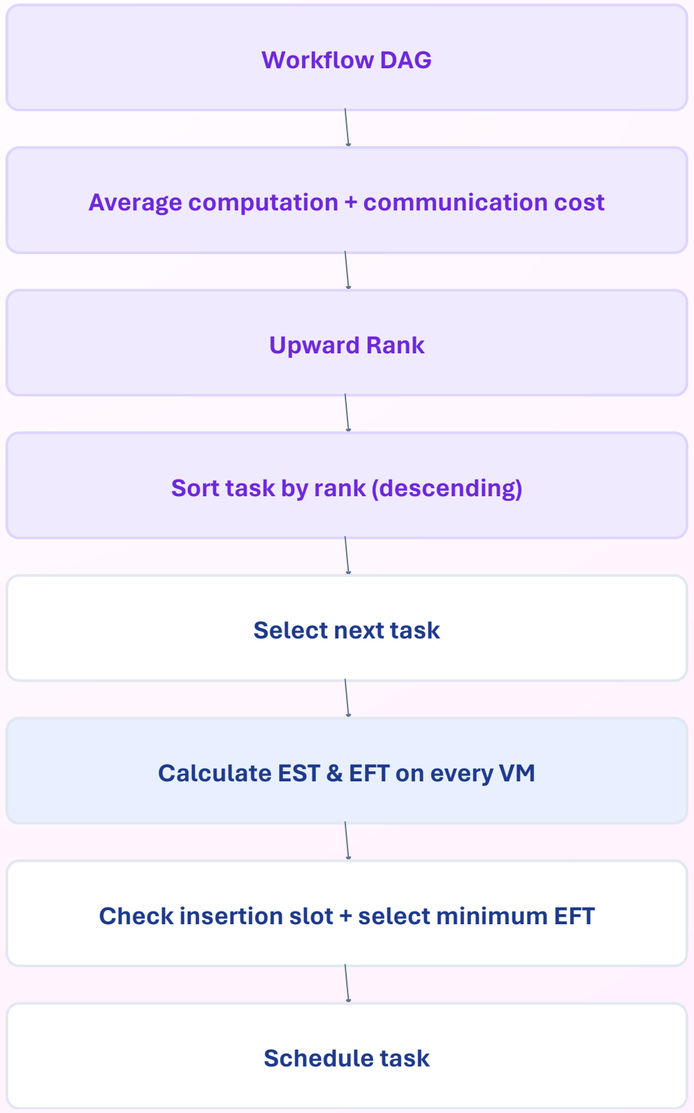
Manfaat

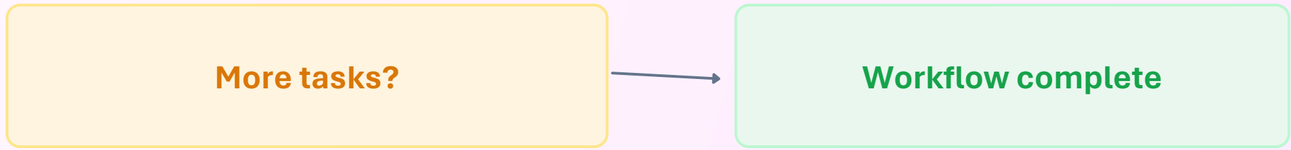
Memanfaatkan idle slot sehingga task tidak selalu harus menunggu antrean paling belakang.

Complete HEFT Flow

Dari workflow DAG sampai final schedule

END-TO-END





YES → kembali ambil task berikutnya

NO → final mapping + timeline

Dua keluaran penting

1. Task → VM mapping
2. Start time / finish time setiap task

→ dari sini makespan dapat dihitung.

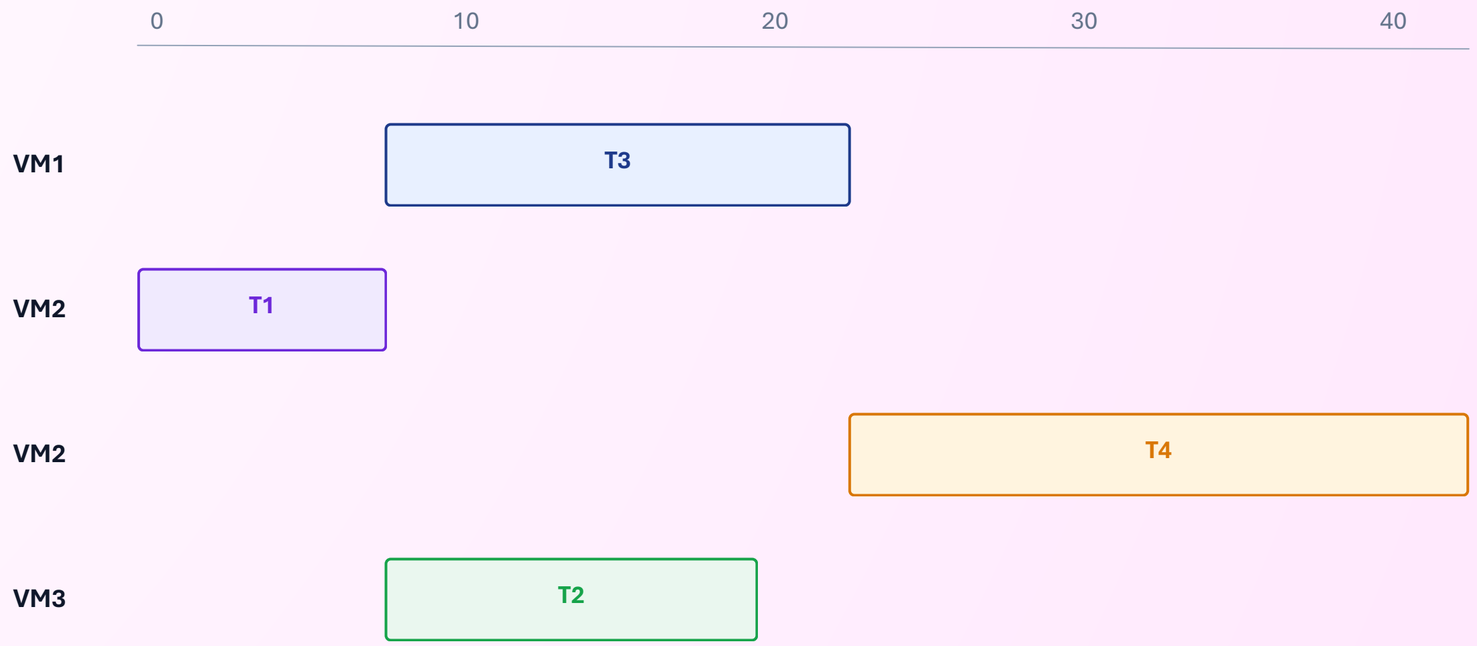
Contoh Final Scheduling

Output HEFT setelah seluruh task dipetakan ke VM

OUTPUT

Task	VM	Start	Finish
T1	VM2	0	8
T2	VM3	8	20
T3	VM1	8	23
T4	VM2	23	43

Gantt chart



Makespan = max(finish time) = 43

Output ini belum merupakan hasil simulator; ini ilustrasi bagaimana schedule HEFT terbentuk.

Apa yang Akan Dievaluasi?

Koneksi HEFT dengan objective dan metric dari Tugas 3

EVALUATION

Makespan

Primary scheduling outcome:
Cmax = max finish time

Energy Consumption

Diukur dari power × time selama execution.

Resource Utilization

CPU dan memory utilization selama workflow.

Scheduling Runtime

Waktu komputasi scheduler untuk menghasilkan mapping.

Dependency Constraint

Child tidak boleh mulai sebelum parent selesai + communication.

Resource Capacity

Tidak boleh melebihi kapasitas VM/host.

Catatan penting HEFT klasik meminimalkan earliest finish time / makespan. Energy pada Tugas 3 menjadi metric evaluasi; menjadikannya objective langsung memerlukan extension energy-aware.

HEFT → CloudSim / WorkflowSim

Posisi Tugas 4 sebelum masuk ke tahap simulasi

NEXT STEP

Tugas 3
Desain
workload + cloud

→

Tugas 4
Pahami +
implement HEFT

→

Tugas 5
Simulasi
di CloudSim

Input eksperimen yang sudah ditentukan

- Synthetic DAG: 100 / 250 / 500 task
- Montage
- CyberShake

Resource scenario

- 2 Datacenter
- 8 Physical Host
- 10 / 20 / 40 VM

Di tahap simulasi

- Implement scheduler
- Jalankan workflow
- Kumpulkan metrics

HEFT menghasilkan task-to-VM mapping; simulator mengeksekusi mapping tersebut dan menyediakan data untuk evaluasi.

Referensi Utama

Sumber primer untuk definisi dan langkah algoritma HEFT

REFERENCE

Topcuoglu, H., Hariri, S., & Wu, M.-Y. (2002).

“Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing.”

IEEE Transactions on Parallel and Distributed Systems, 13(3), 260–274.

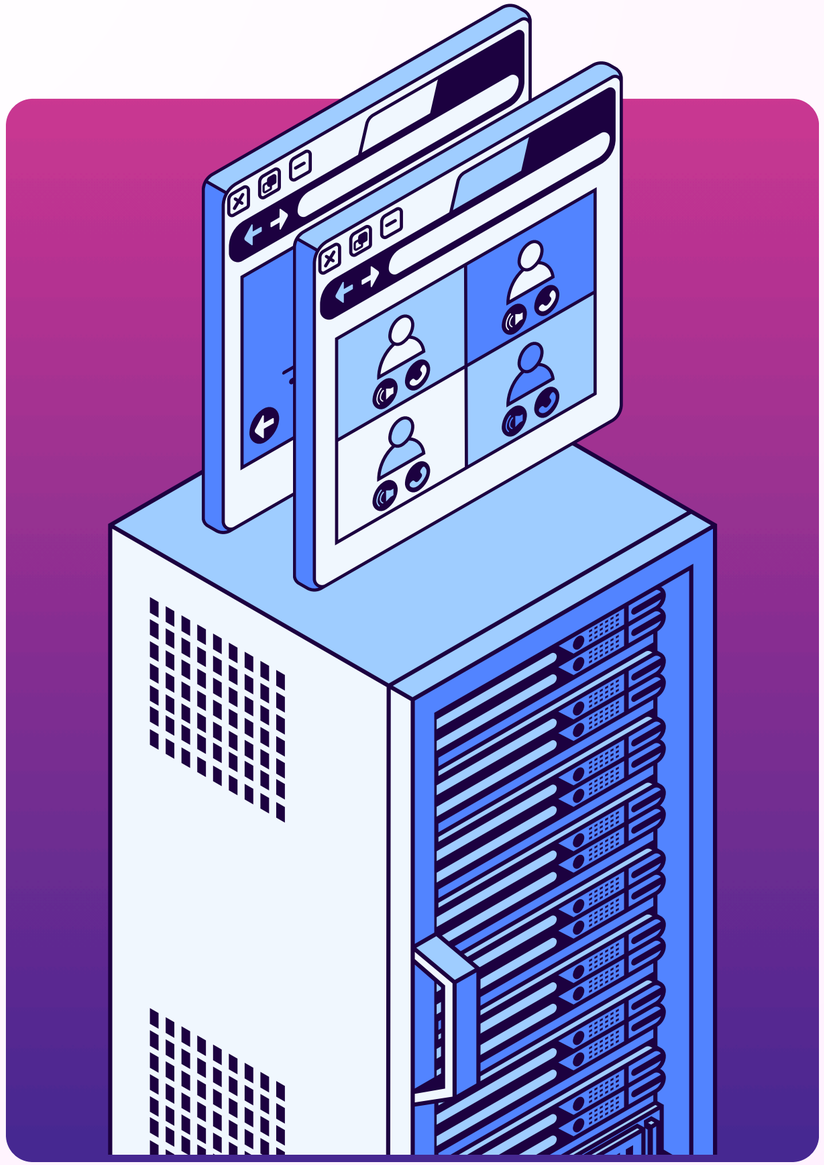
DOI: 10.1109/71.993206

https://doi.org/10.1109/71.993206

Konsep yang dirujuk di deck

Upward rank • processor selection • earliest finish time • insertion-based scheduling

HEFT adalah baseline heuristic yang akan dibawa ke tahap implementasi dan simulasi berikutnya.



Let’s build **Your Next Breakthrough.**

hello@reallygreatsite.com